

Good Coding Practices in Economics

Jakub Przewoski

Economic Society for Bocconi Students

March 2026

Today's Agenda

- ① Introduction
- ② Setup
- ③ Writing Good Code
- ④ Git

Why Does This Matter?

- Code is **mostly read, not written** — by colleagues, supervisors, and your future self
- Open-source languages and conventions are a long-term investment in efficiency
- Today: a quick overview of core concepts + a **live demo**

The Golden Rule

Always code as if the person who will maintain your code is a violent psychopath who knows where you live.

What Is Reproducibility?

Scenario: A professor asks you to write code he can run on his machine in one click. Later, he'll adjust it and eventually take over the project entirely.

What should your code guarantee?

- **Dependency transparency** — what libraries does it need, and which versions?
- **Documentation** — can someone understand it quickly without asking you?
- **Modularity** — can individual parts be changed without breaking everything else?

The core goal

Anyone with your code and your instructions should get *exactly* the same results.

R, Python, or Stata?

Stata

- Easy to learn
- Limited scope
- Requires a license
- Rare outside academia

R

- Excellent for statistics
- Rich econometrics ecosystem
- Less general-purpose
- Free & open source

Python

- General purpose
- ML, simulation, data
- Stat inference improving
- Free & widely used

Disclosure

Today we will be working with Python, but the lessons here are language-agnostic. IRL, it's probably best if you learn a bit of all three.

The Tools You Need

Language & Package Manager

- **Miniconda** — lightweight, terminal-based (recommended)
- **Anaconda** — includes a GUI, easier to start with
- **R** from CRAN: `cran.r-project.org`

Code Editor

- **VS Code** — lightweight, highly customizable, free
- Open it through the terminal or project folder to set up a workspace

Conda Environments

The problem: Different projects may need different (sometimes conflicting) package versions.

The solution: A **conda environment** is an isolated, self-contained Python installation per project.

- Each project gets its own packages and versions
- Environments don't interfere with each other
- Easy to share: export to `environment.yml`
- **R alternative:** The `renv` package

```
# create
conda create -n myproject python
              =3.11
# activate
conda activate myproject
# export
conda env export > environment.yml
```

VS Code Extensions

Activate extensions *per workspace* to keep startup time fast and battery usage low.

Must-haves

- **Python** — linting, autocomplete, IntelliSense
- **Jupyter** — run notebooks and interactive `.py` mode inside VS Code
- **Data Wrangler** — visual data viewer

Nice to have

- **GitHub Copilot** — free with GitHub Education
- **LaTeX Workshop** — write papers locally inside VS Code
- **R extension** — if you use R

A Clean Project Folder

Good structure means anyone (including future-you) can navigate the project in seconds.

```
my_project/  
|-- data/  
|   |-- raw/  
|   |-- clean/  
|-- src/  
|   |-- 01_clean.py  
|   |-- 02_analysis.py  
|   |-- __main__.py  
|-- output/  
|   |-- figures/  
|   |-- tables/  
|-- writing/  
|-- README.md  
|-- environment.yml
```

- **Raw data is never modified** — always save a clean copy separately
- **README.md** is not optional — it's the front door to your project
- Keep one **sample project folder** and copy it every time you start something new

The Essentials

- **Separate files for separate tasks** — cleaning, analysis, and a `__main__.py` that ties it together
- **DRY — Don't Repeat Yourself** — if you copy-paste code more than twice, write a function
- **Comment the *why*, not the *what*** — the code already says what it does; explain your reasoning
- **Write docstrings** — a few lines above each function describing inputs, outputs, and purpose
- **Name variables consistently** — `gdp_growth_2023`, not `x2` or `temp_var_final_v3`
- **Save intermediate clean data** — avoid re-running expensive cleaning steps on every run

Comments & Docstrings

Bad comment — states the obvious:

```
# multiply x by 2  
result = x * 2
```

Good comment — explains the why:

```
# Annualize monthly growth rate  
# using compound formula  
result = (1 + x)**12 - 1
```

A docstring:

```
def clean_cpi(df, base_year):  
    """  
    Rebase CPI series to a given  
    base year.  
  
    Parameters:  
        df: pd.DataFrame with '  
            date'  
            and 'cpi' columns  
        base_year: int  
  
    Returns:  
        pd.DataFrame with rebased  
        CPI  
    """
```

Useful Tricks

Interactive mode in VS Code

- Use Shift + Enter to run your .py file like a notebook (run code block-by-block)
- Great for exploratory analysis while keeping your code in proper scripts

Logging & resumable pipelines

- When developing the code, use the `print()` statements to test it,
- Before running the code, switch to Python's logging module writes different statements (e.g. errors) to a file

AI tools

GitHub Copilot is excellent for boilerplate and docstrings. Use it as a *first draft* — always read and understand what it writes (or use Claude Code instead).

What Is Git?

Git is a version control system — it tracks every change you make to your code over time.

What it lets you do:

- See exactly what changed between versions and when
- Revert to any previous state if something breaks
- Experiment safely using **branches** — a separate line of development that won't touch your working code
- Collaborate with others and **merge** changes together
- See who wrote which line and why (commit messages)

Think of it as ...

Track Changes in Word, but for your entire project — and you control exactly when a snapshot is taken.

Git: The Basic Workflow

```
# Start tracking a project
git init

# Stage your changes
git add .

# Save a snapshot
git commit -m "Add CPI cleaning
script"

# See what changed
git diff

# See your history
git log --oneline
```

- **GitHub** hosts your repository online — backup + collaboration
- **GitHub Education** gives you free Pro access (private repos, Copilot, and more) with your university email
- Always include a `.gitignore` to avoid committing data files, credentials, or large outputs

Branches: Experiment Safely

The idea

A branch is a parallel version of your project. Make changes there freely — your main code stays untouched until you decide to merge.

```
# Create and switch to a new
  branch
git checkout -b new-specification

# ... make changes, commit ...

# Merge back when ready
git checkout main
git merge new-specification
```

Useful for:

- Testing a different model specification
- Trying a different data cleaning approach
- Responding to reviewer comments without touching working code

Takeaways

- ① **Reproducibility** is not optional in research — document dependencies, structure, and reasoning
- ② **Python and R** are worth learning; they'll serve you in academia and industry
- ③ **Environments** keep projects isolated and shareable
- ④ **Structure** your project before writing a single line of code
- ⑤ **Write for readers** — comment the why, use functions, keep it DRY
- ⑥ **Use Git** — you will eventually break something, and you'll be glad you did

Further Reading

- **Missing Semester** (MIT) — missing.csail.mit.edu
Practical tools every developer should know: shell, git, editors, and more
- **Coding for Economists** — aeturrell.github.io/coding-for-economists
Reproducible analysis and best practices tailored for economics research
- **Why Every Economics Student Should Use GitHub**
econ-jobs.com — motivation and practical starting points

Writing Papers Locally with L^AT_EX

Why bother with local L^AT_EX instead of Overleaf?

Advantages

- Figures and output live in the same project — no manual transfers
- No compile limits, full privacy, works offline
- Integrates with GitHub for version control of your writing
- Zotero integration auto-updates your bibliography
- AI completions via Copilot

Disadvantages

- Initial setup takes some effort
- Debugging (e.g. bibliography issues) requires more patience
- Real-time collaboration is less smooth than Overleaf

VS Code extensions:

- **LaTeX + LaTeX Workshop**